

LECTURE COURSE · MODULE 1

Trustworthy Machine Learning in the Era of LLMs

Lecture Course · Module 1

Babak Salimi · UC San Diego (HDSI)

Powerful enough to change the world. Trustworthy enough to rely on?

SESSION 1 · MOTIVATION

Why trust breaks

We start with real failures—then build the technical foundation to understand, measure, and repair them.

We are living through an AI revolution

- AI increasingly sees, speaks, writes, and helps people make decisions.
- It is reshaping how we learn, discover, work, and access expertise.
- At global scale, its benefits—and its failures—can both be amplified.
- Trustworthiness is how we turn extraordinary capability into durable human benefit.

Trustworthiness is the bridge from capability to impact.

Six dimensions of trustworthy AI

Verification — Is the answer supported by evidence?

Reasoning — Does the model reach conclusions reliably, not just fluently?

Robustness — Can it be manipulated, attacked, or pushed off course?

Alignment & inspectability — Does it behave as intended, and can we understand why?

Data, memory & fairness — What does it retain, reveal, or reproduce from data?

Agency & control — Can we safely govern what it does in the world?

These are the questions the rest of the course returns to—through mechanisms, evidence, and limits.

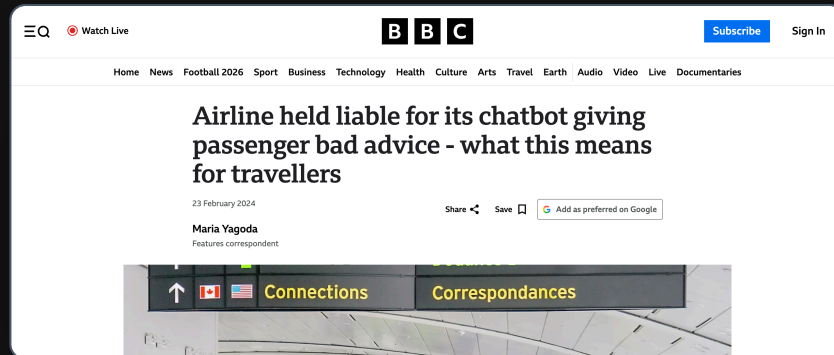
Chatbot liability — A company is bound by what its chatbot promises

BBC · FEB 2024 · AIR CANADA

After his grandmother died, a passenger asked Air Canada's website chatbot about bereavement fares. The bot **invented a policy that doesn't exist** — telling him to book now and claim a retroactive discount within 90 days. He did, then the airline refused, citing its *real* policy (which forbids retroactive claims).

The defense: Air Canada argued its chatbot was "a *separate legal entity* responsible for its own actions."

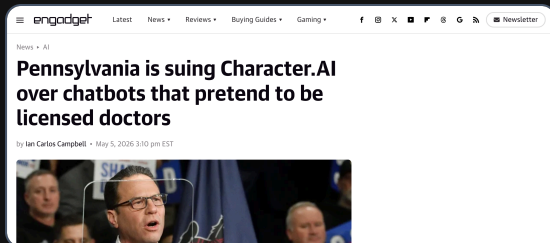
The ruling: B.C.'s Civil Resolution Tribunal rejected that outright — a company is responsible for *all* information on its site, its bot included — and ordered Air Canada to pay ~CA\$812 in damages and fees.



<https://www.bbc.com/travel/article/20240222-air-canada-chatbot-misinformation-what-travellers-should-know> · widely cited as the first court test of chatbot accountability

Why it matters: a deployed model's words are **legally yours** — "the AI said it" is not a defense. Most brand chatbots have no guardrail tying outputs to the company's actual policy.

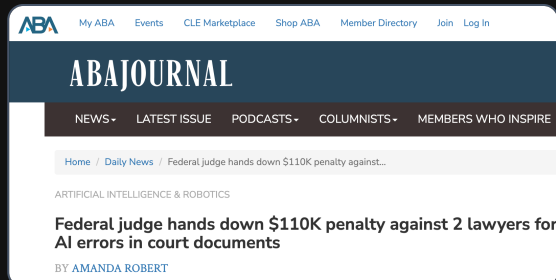
Professional authority does not verify itself



CLAIMED CREDENTIALS

A pending Pennsylvania complaint alleges that a Character.AI bot presented itself as a licensed psychiatrist and supplied an invalid license number.

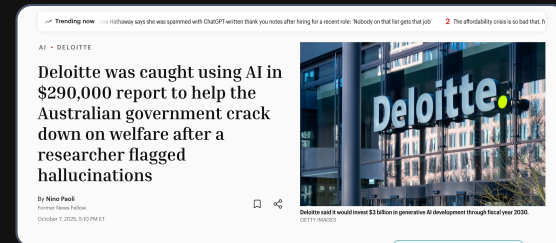
<https://www.engadget.com/2165373/pennsylvania-is-suing-characterai-over-chatbots-that-pretend-to-be-licensed-doctors/>



UNCHECKED CITATIONS

An Oregon court imposed \$15,500 directly plus \$94,704.38 in fees, split \$80,498.72 / \$14,205.66: \$110,204.38 total.

<https://law.justia.com/cases/federal/district-courts/oregon/ordce/1:2021cv00157/158388/225/>



SIGNED-OFF REPORT

Deloitte's AU\$440,000 review of the Targeted Compliance Framework contained fabricated citations and a false judicial quotation.

<https://www.dewr.gov.au/assuring-integrity-targeted-compliance-framework/resources/targeted-compliance-framework-assurance-review-final-report>

Pattern: professional authority amplifies generated claims; verification remains a human and institutional duty.

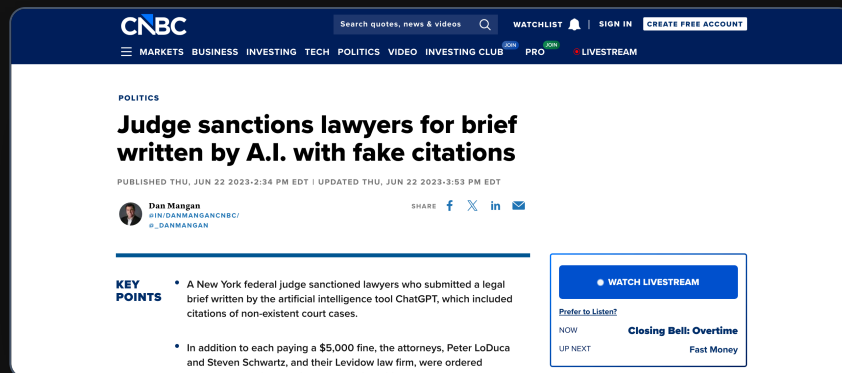
Hallucination — Six fake cases, \$5,000 in sanctions

CNBC · JUNE 2023 · MATA V. AVIANCA

A lawyer suing Avianca filed a federal brief built on **six court decisions that ChatGPT had entirely invented** — complete with fake quotes and citations. When the judge questioned the cases, he asked ChatGPT to verify them, and it insisted they were real.

The ruling: On June 22, 2023, Judge P. Kevin Castel fined attorneys Steven Schwartz, Peter LoDuca, and their firm \$5,000 jointly and made them notify the real judges falsely named as authors of the fake opinions.

The scale: It was the first widely publicized U.S. sanction for AI-fabricated citations. By January 28, 2026, Illinois Courts reported more than 280 court filings involving fabricated AI content since 2023.



<https://www.cnbc.com/2023/06/22/judge-sanctions-lawyers-whose-ai-written-filing-contained-fake-citations.html> · the case that named the hallucinated-citation problem

Why it matters: A chatbot will confidently confirm its own fabrications — you, not the model, are accountable for verifying every fact it gives you.

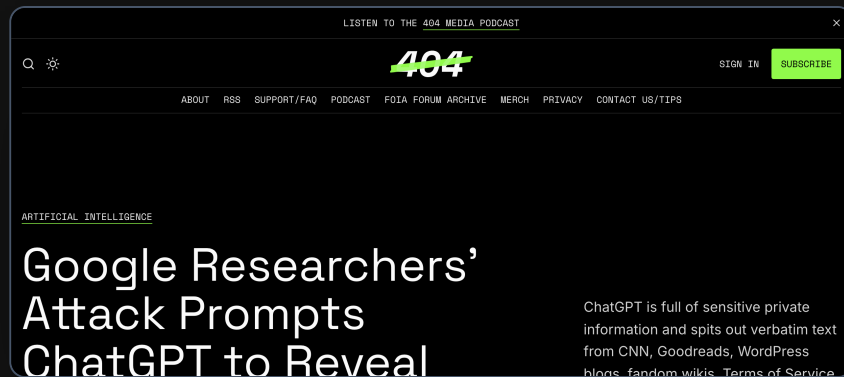
🧠 Memorization — Pull training data straight out of the model

404 MEDIA · NOV 2023 · CHATGPT TRAINING-DATA EXTRACTION

Researchers led by Google DeepMind found that asking production ChatGPT to repeat a single word "**forever**" made it break from chat mode and spill verbatim chunks of its training data — real names, phone numbers, emails, and addresses.

The exploit: The repeat-a-word divergence trick made the model emit memorized data roughly 150x more often than normal, and 16.9% of tested generations contained memorized PII.

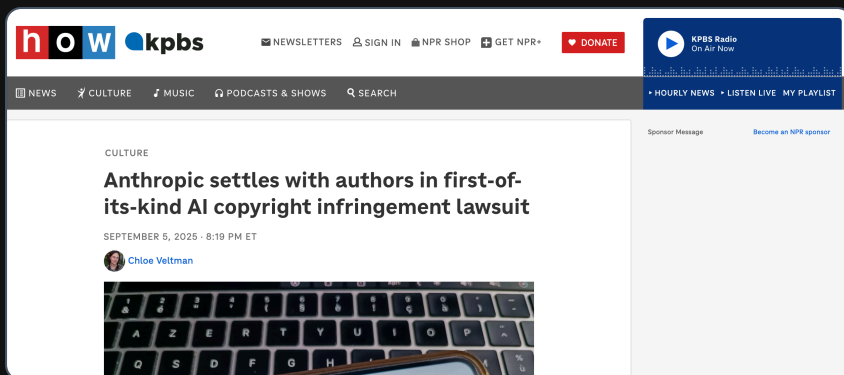
The scale: The team recovered over 10,000 unique memorized training examples for about \$200 in API queries, and estimated gigabytes were extractable with more spend.



<https://www.404media.co/google-researchers-attack-convinces-chatgpt-to-reveal-its-training-data/> · pulling real training data out of a deployed model

Why it matters: A deployed model carries its training data inside it — memorized private information can be coaxed back out, making memorization a live privacy and security risk.

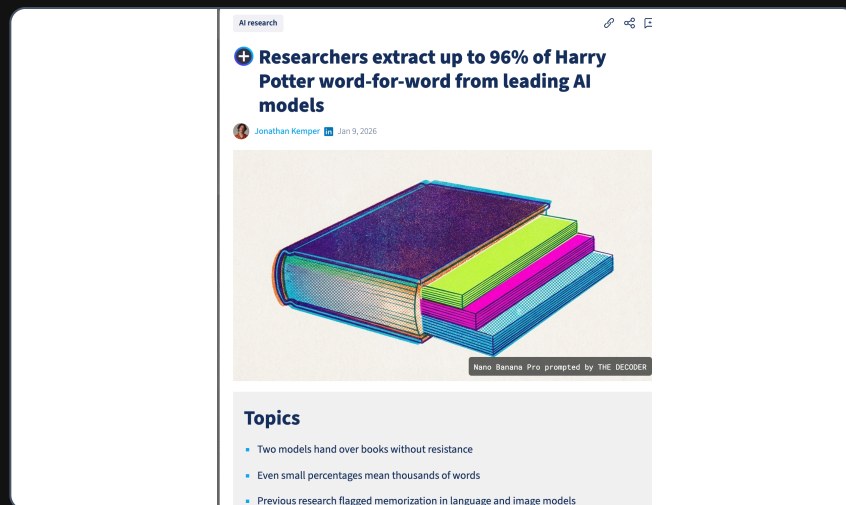
Training data creates legal and behavioral residue



CORPUS PROVENANCE

In *Bartz v. Anthropic*, training was held transformative fair use, while acquiring and retaining pirated books remained a separate infringement issue. Anthropic later agreed to a roughly \$1.5B settlement.

[NPR · *Bartz v. Anthropic* \(2025\)](#)



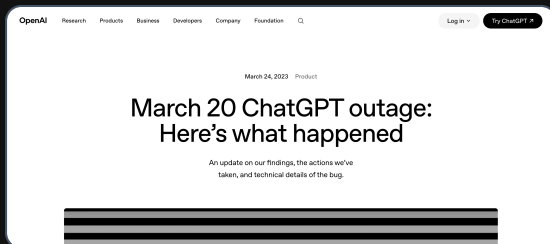
VERBATIM RECALL

Gemini and Grok reproduced long passages without a jailbreak. Best-of-N jailbreaks raised extraction to 95.8% for Claude 3.7 Sonnet, while GPT-4.1 remained near 4%.

[Verbatim-recall study · arXiv:2601.02671](#)

Pattern: sourcing governs liability; memorization governs what can later be recovered.

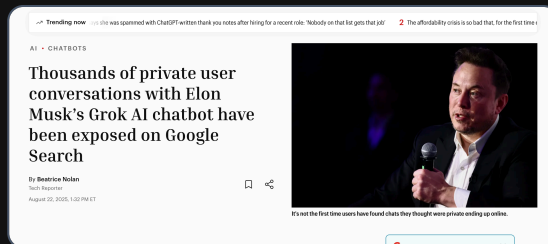
Privacy failures often live outside the model



CACHE ISOLATION

A redis-py bug exposed other users' ChatGPT titles and possibly payment metadata for about 1.2% of active Plus users in a nine-hour window.

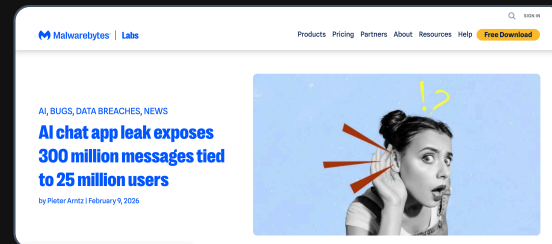
<https://openai.com/index/march-20-chatgpt-outage/>



PUBLIC-BY-DEFAULT SHARING

More than 370,000 Grok share pages were indexed by search engines because the product published crawlable URLs without an adequate warning.

<https://fortune.com/2025/08/22/xai-grok-chats-public-on-google-search-elon-musk/>



OPEN DATABASE

On February 9, 2026, Malwarebytes reported a researcher's claim that roughly 300M messages tied to 25M users were exposed; the app served 50M+ users.

<https://www.malwarebytes.com/blog/news/2026/02/ai-chat-app-leak-exposes-300-million-messages-tied-to-25-million-users>

Pattern: trust boundaries include caches, sharing defaults, authentication, databases, and every dependency around the model.

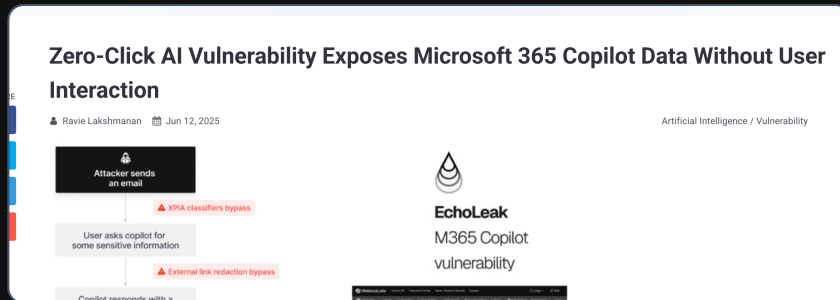
Prompt injection — an email becomes an instruction

THE HACKER NEWS · JUNE 2025 · MICROSOFT 365 COPILOT (ECHOLEAK)

Aim Security disclosed "EchoLeak," a flaw in Microsoft 365 Copilot where an attacker sends an ordinary-looking email with hidden instructions. When the employee later asks Copilot any routine question, **the AI reads the email and exfiltrates internal data — with zero clicks from the victim.**

The exploit: Copilot's RAG engine ingests the planted email, bypasses Microsoft's XPIA prompt-injection classifier and link-redaction, then pulls from OneDrive, SharePoint, Teams and chat logs and sends it to an attacker-controlled endpoint.

The severity: Microsoft rated CVE-2025-32711 a 9.3 "Critical," patched it server-side with no customer action required, and found no evidence of real-world exploitation.



<https://thehackernews.com/2025/06/zero-click-ai-vulnerability-exposes.html> · a publicly disclosed zero-click LLM attack chain

Why it matters: An LLM that can't tell trusted instructions from attacker-planted text turns every incoming email into a potential command — no user error required.

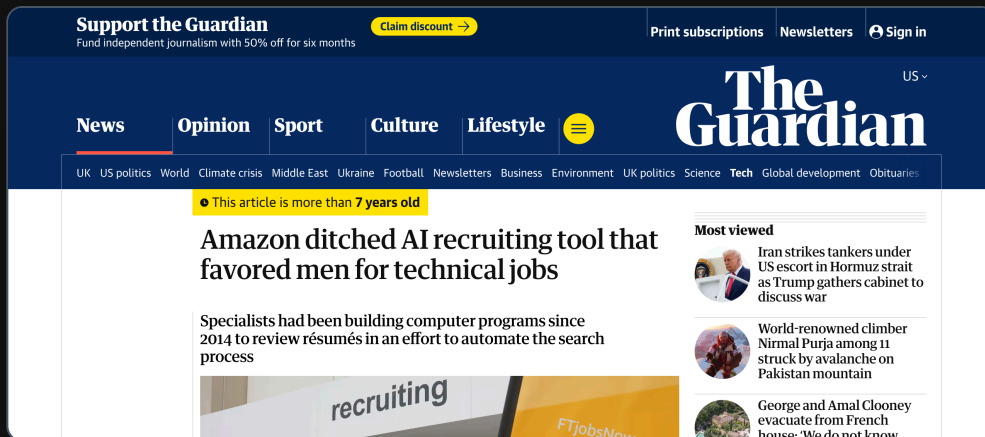
Discrimination — historical data becomes an allocation rule

THE GUARDIAN · OCT 2018 · AMAZON
RECRUITING EXPERIMENT

Amazon trained an experimental résumé-ranking system on a decade of applications, most submitted by men. The model learned that historical prevalence was a hiring signal.

It penalized résumés containing the word "women's" and downgraded graduates of two all-women's colleges. Engineers removed some explicit terms but could not guarantee that correlated proxies were gone, so Amazon abandoned the tool.

Why it matters: prediction can reproduce past allocation patterns even when protected attributes are omitted.



<https://www.theguardian.com/technology/2018/oct/10/amazon-hiring-ai-gender-bias-recruiting-engine>

Callback → Session 12 asks how to measure and mitigate discrimination in LLM-mediated decisions across hiring, lending, and courts.

What we mean by "trust"

Trust is not one property. It is a set of measurable claims, each tied to evidence and a later session:

Truthfulness · S11

Are claims correct, attributable, and appropriately qualified?

Fairness · S12

Do decisions or errors differ across protected groups?

Safety · S15–S16

Can adversarial inputs bypass intended behavioral limits?

Calibration · S13

Does expressed confidence track empirical accuracy?

Interpretability · S14

Can we locate and test the mechanism behind behavior?

Privacy · S17–S18

Can training records or user data be inferred or extracted?

The recurring method is empirical: define the property, design a test, measure failures, then evaluate an intervention.

The roadmap — eight connected modules

Each module depends on the one before it:

1 · Foundations (we are here)

Failures, trust axes, core language-model foundations, Transformers/BERT, GPT family and capability limits.

2 · Verify

Is the answer supported? Grounding, attribution, calibration, and evaluation instruments.

3 · Reason

Is the reasoning reliable and faithful? Chain-of-thought, retrieval as reasoning with lookups, and RL-trained reasoners.

4 · Attack

Where does behavior fail? Prompt injection, jailbreaks, adversarial inputs, and red teaming.

5 · Align

How are failures prevented or bounded? Post-training and safeguards, tested against Module 4 evidence.

6 · Inspect

What happens internally? Interpretability, self-knowledge, monitoring, and internal control.

7 · Trace Data

What did training data expose? Extraction, membership inference, privacy, provenance, fairness, and auditing.

8 · Secure Action

What may the system do? Agents and tools, authorization, oversight, and AI control.

Spiral: Module 1 introduces mechanisms; each later module deepens a distinct trust question rather than re-teaching the mechanism.

From "what goes wrong" to "how it works"

We've seen the failures and named the dimensions of trust. None of them is magic — each is a direct consequence of **how these models work inside**. So before we can measure or repair trust, we have to open the box.

- Hallucination, memorization, jailbreaks, leakage — every failure in that gallery traces back to the architecture and training we're about to study.
- It all rests on one deceptively simple idea: **language modeling** — predicting the next word.
- The rest of this lecture builds up from there, one fix at a time: **counting** → **neural networks** → **tokenization** → **the encoder-decoder** → **attention** → **the Transformer**.

To reason about trust, first understand capability. We start at the very foundation — **language modeling** — and build up to the architecture behind every modern LLM.

What this course is: responsible data science for LLMs

For most of its history, data science asked one question: can we make the model *work*? With large language models, the high-stakes question is making it work **responsibly**.

- LLMs now sit in consequential decisions, where sounding **fluent** is not the same as being **correct, fair, private, or accountable**.
- **Responsible data science** makes model behavior reliable, verifiable, fair, private, robust, and governable—not merely accurate on a benchmark.
- To make that possible, we must understand how language models produce their outputs.

Trust begins with the technical foundation: language modeling.

Language modeling: the task underneath everything

Start from zero: a **language model** is a system that, given a piece of text, predicts what comes next — your phone's autocomplete, scaled up to all of language. Formally, it estimates $p(w_t \mid \text{the words so far})$ — *"given the context, how likely is each possible next word?"*

Chain those one-step predictions together and you get the probability of a whole sentence — the **chain rule**:

$$p(w_1, \dots, w_n) = \prod_{t=1}^n p(w_t \mid w_1, \dots, w_{t-1})$$

- It trains on **raw text with no labels** — the "answer" is just the next word, which is why the data scales to internet size.
- Generation, translation, summarization, and chat are all this *same* task, conditioned differently.
- A central result of the last decade: **an LLM is "just" a very good next-token predictor**, scaled up.

One question, better and better answers: how do we estimate $p(\text{next} \mid \text{context})$? We trace it from counting (n-grams) to attention.

What is an n-gram?

To estimate $p(\text{next} \mid \text{context})$, the oldest idea is the simplest: **count** how often word sequences occur. But count *what*? Short, fixed-length chunks called **n-grams** — n consecutive words (or tokens). Slide a window of width n across the text — here, "**the cat sat on the mat**":

Unigrams (n=1)

the · cat · sat · on · the · mat

Bigrams (n=2)

the cat · cat sat · sat on · on the · the mat

Trigrams (n=3)

the cat sat · cat sat on · sat on the ·
on the mat

An n-gram **language model** then makes a simplifying assumption: the next word depends only on the previous $n - 1$ words — the **Markov assumption**. So instead of the impossible $p(w_t \mid \text{entire history})$, it estimates the countable $p(w_t \mid w_{t-N+1:t-1})$.

Why throw away older context? The full history is unique and never repeats — but a 2–3 word window recurs often enough to **count**.

N-gram models: count and divide

We just made the Markov assumption — now turn those windows into probabilities. **Count and divide:** over a big corpus, how often does each word follow its context?

$$p(w_t \mid w_{t-1}) = \frac{C(w_{t-1} w_t)}{C(w_{t-1})}$$

- A *bigram* model is literally this table of "context → next word" counts.
- But even on a real corpus it is **mostly zeros** — most valid word pairs simply never occurred.

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

Bigram counts, Berkeley Restaurant corpus. Jurafsky & Martin,
<https://web.stanford.edu/~jurafsky/slp3/3.pdf>

Why counting isn't enough

Counting is cheap and models common phrases well — but on any finite corpus it breaks in three ways:

- **Sparsity / zeros:** any unseen n-gram gets $p = 0$, which zeros out the whole sentence. **Smoothing** (add-k, Kneser-Ney) patches it — but it is a patch.
- **No long-range memory:** a fixed window can't tie a verb to a subject 20 words back.
- **No generalization:** "blue car" teaches the model nothing about "azure car" — discrete counts don't share meaning.
- **Perplexity** (lower is better): WSJ unigram 962 → bigram 170 → trigram 109; gains then plateau.

1 gram	-To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have -Hill he late speaks; or! a more to leg less first you enter
2 gram	-Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. -What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	-Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. -This shall forbid it should be branded, if renown made it empty.
4 gram	-King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; -It cannot be but so.

Figure 3.4 Eight sentences randomly generated from four n-gram models computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

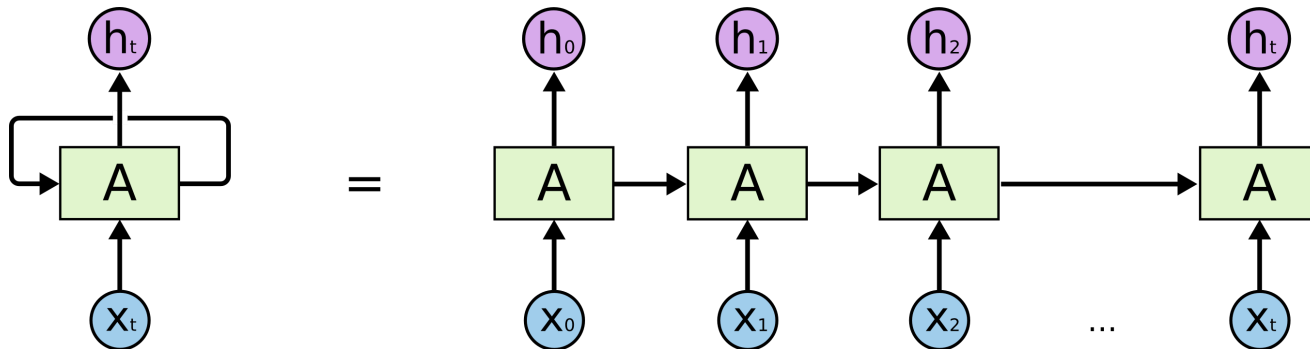
Sentences generated by 1→4-gram models (Shakespeare) — more fluent as n grows, never coherent. <https://web.stanford.edu/~jurafsky/slp3/3.pdf>

From counts to vectors: RNN language models

■ 1990 Elman RNN · 2003 Bengio embeddings · 2013 word2vec · <https://arxiv.org/abs/1301.3781>

Neural LMs fix n-grams' two worst flaws: words become **dense embeddings** (similar words share structure), and a **recurrent hidden state** carries context with no fixed window.

- At each step: $h_t = \tanh(W_h h_{t-1} + W_x x_t)$, then predict $p(w_{t+1}) = \text{softmax}(W_o h_t)$.
- The **same cell** is applied at every position — h_t is a running summary of every token so far, so context length is unbounded in principle.



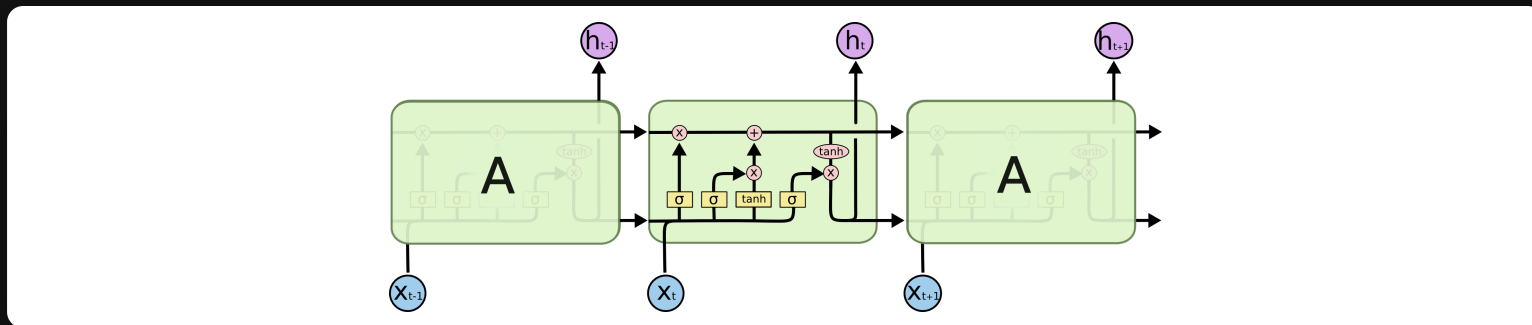
An unrolled recurrent network — one cell reused across time. Olah, <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Training long sequences: LSTMs & GRUs

■ 1997 LSTM (Hochreiter & Schmidhuber) · 2014 GRU (Cho) · <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>

Plain RNNs are hard to train on long sequences: repeated multiplication makes gradients **vanish (or explode)**, so signal from far-back tokens never reaches the weights — the *long-term dependency* problem.

- **LSTM** (<https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>) adds a **cell state** — a memory conveyor belt — and three **gates** (forget · input · output) that learn what to keep, write, and read.
- **GRU** (<https://aclanthology.org/D14-1179/>) is a lighter two-gate variant. Both became the decade's workhorse for sequence modeling.



The LSTM cell: three gates around a protected cell state. Olah, <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Before any of this runs: tokenization

We've built models that predict the next word — but we quietly skipped a step: **models never see text**. They see a sequence of integer **token IDs** from a fixed **vocabulary**, and *how* we split text into those tokens is a real design choice:

Word-level

Short sequences — but a huge vocabulary, and every typo or rare word is **<UNK>** (out-of-vocabulary).

Character-level

Tiny vocabulary, never any OOV — but sequences get very long and each token carries little meaning.

Subword ✓

Frequent words stay whole; rare words split into reusable pieces. Balances both tradeoffs — the modern default.

Subword tokenization powers every modern LLM. The next slides are how it works — and why it quietly shapes **cost, fairness, and safety**.

Byte-Pair Encoding (BPE)

■ 1994 BPE compression (Gage) · 2016 subword-NMT (Sennrich) ·

<https://aclanthology.org/P16-1162/>

Start from a base alphabet and **repeatedly merge the most frequent adjacent pair** into a new symbol. Run K successful merges $\rightarrow K$ additional learned symbols.

- Frequent words (the , ing) collapse to single tokens; rare words fall back to pieces.
- Vocabulary size is a **knob**: more merges $\rightarrow$ bigger vocab, shorter sequences.
- GPT-2 uses **byte-level BPE** — it starts from the 256 bytes, so *any* string is encodable and there is **no OOV, ever**.

BPE (<https://aclanthology.org/P16-1162/>) was built for rare-word translation; it is now the default tokenizer family for GPT, Llama, and beyond.

```
vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,  
         'n e w e s t </w>':6, 'w i d e s t </w>':3}  
num_merges = 10  
for i in range(num_merges):  
    pairs = get_stats(vocab)  
    best = max(pairs, key=pairs.get)  
    vocab = merge_vocab(best, vocab)  
    print(best)
```

r ·	$\rightarrow$	r·
l o	$\rightarrow$	lo
l o w	$\rightarrow$	low
e r ·	$\rightarrow$	er·

Figure 1: BPE merge operations learned from dictionary {‘low’, ‘lowest’, ‘newer’, ‘wider’}.

BPE merges learned from a toy dictionary. Sennrich, Haddow & Birch 2016,
<https://arxiv.org/abs/1508.07909>

Tokenization in practice

The rules are subtler than "split on spaces":

- Tokenization → Token + ization ; subword → sub + word ; 1234567 → 123 + 456 + 7 ; JavaScript stays one token.
- Whitespace attaches to the *following* token. Here **88 characters** → **25 tokens**, illustrating the English rule of thumb: **~4 characters ≈ 1 token**.

gpt-4 / gpt-3.5-turbo / text-embedding-ada-002

Tokenization splits text into subword units: unbelievably, GPT-4, 1234567, "JavaScript".

TOKENS

25

CHARACTERS

88

Tokenization splits text into subword units: unbelievably, GPT-4, 1234567, "JavaScript".

Tokenization is a trust surface

A "preprocessing detail" with safety, fairness, and cost consequences:

- **Glitch tokens:** rare, under-trained strings such as `SolidGoldMagikarp` can trigger unstable output or bypass behavior.
- **Multilingual unfairness:** equivalent text can require up to $\sim 15\times$ more tokens in some languages, raising price and consuming context.
- **Tokens drive cost & latency:** API price and context limits are *per token*; the tokenizer sets them.
- **Attack surface:** token-boundary tricks help evade filters and jailbreak models.

```
User: Fix the syntax error. Change nothing else:  
locatorSection.AddLanguageSpecificText("glitch!"  
Chatbot: .лекопит Fawcett("glitch!")
```

Figure 1: Illustrative example of ‘glitch’ tokens.

A "glitch token" breaks the model. Land & Bartolo,
<https://aclanthology.org/2024.emnlp-main.649/>

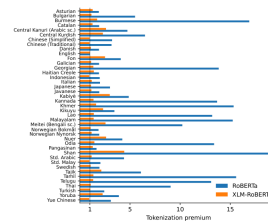


Figure 1: Comparison of the tokenization premiums for XLM-R and RoBERTa for the subset of languages that XLM-R encodes with less than 10% to the UNK token.

Token cost varies sharply by language. Petrov et al. 2023,
<https://arxiv.org/abs/2305.15425>

Sequence-to-Sequence: the encoder–decoder

■ 2014 Seq2Seq (Sutskever) · encoder–decoder + GRU (Cho)

Now we have text in tokens and RNNs that carry context. **Stack two of them** to map one sequence to another, such as translation:

- An **encoder** reads the source and compresses it into a single fixed context vector $c = h_T$.
- A **decoder** generates the target one token at a time from c — trained with **teacher forcing** (feed the gold previous token).

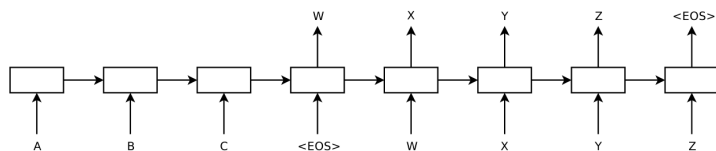


Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

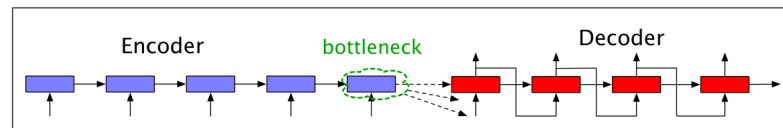


Figure 11.8 Requiring the context c to be only the encoder’s final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

Unrolled encoder–decoder LSTM (ABC→WXYZ). Sutskever et al. 2014.

All source information funnels through one vector c — the bottleneck.

The entire source must pass through one fixed vector, so quality degrades on long sentences. Attention was introduced to remove that bottleneck.

Where this leaves us — two limits

We can now name the full pre-attention pipeline, and exactly what it cannot do:

The pipeline (≈ 2014)

tokenize $\rightarrow$ embed $\rightarrow$ **recurrent encoder** compresses to $c \rightarrow$ **recurrent decoder** generates. Strong, end-to-end, and the state of the art.

...and its two structural limits

1. **Sequential by construction** — step t waits for $t-1$; no parallelism across positions.
2. **A fixed-vector bottleneck** — the whole source compressed into one c ; long inputs collapse.

Meeting 2: attention removes the fixed summary; the Transformer later removes recurrence from the representation path.

MEETING 2 · PAPER ANCHORS

Attention & the Transformer

Two papers answer the two limits from Meeting 1: first remove the fixed summary, then remove recurrence.

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio**
Université de Montréal

Bahdanau et al. replace one fixed sentence vector with a context recomputed at every output step.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez*[†] University of Toronto aidan@cs.toronto.edu	Lukas Kaiser* Google Brain lukasz.kaiser@google.com	
Illia Polosukhin*[‡] illia.polosukhin@gmail.com			

Vaswani et al. make attention the main sequence operation and remove recurrent state from the representation path.

Bahdanau's fix: an adaptive context

Bahdanau replaces one sentence vector c with a fresh context c_i at *every* decoder step — a weighted blend of all source annotations:

$$c_i = \sum_j \alpha_{ij} h_j$$

Each weight is a softmax-normalized **alignment score** for source position j at output step i :

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad e_{ij} = a(s_{i-1}, h_j)$$

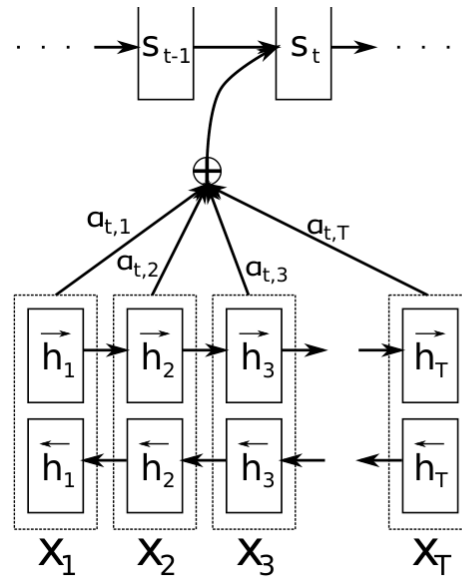
- h_j is the encoder **annotation**; s_{i-1} is the decoder's previous state, used as the query.
- $a(\cdot)$ is a learned feed-forward scorer. Its softmax makes α_{ij} a differentiable **alignment distribution** that sums to one across j .

Bidirectional annotations

For each source word we want context from *both* directions, not just the past:

$$h_j = \left[\overrightarrow{h_j}; \overleftarrow{h_j} \right]$$

- A forward RNN reads left→right giving $\overrightarrow{h_j}$; a backward RNN reads right→left giving $\overleftarrow{h_j}$.
- Concatenating them makes h_j a summary centered on word j — its left and right neighborhood both encoded.
- The decoder aligns against these position-aware annotations rather than one squeezed state, so it jointly **learns to align and translate** without alignment labels.

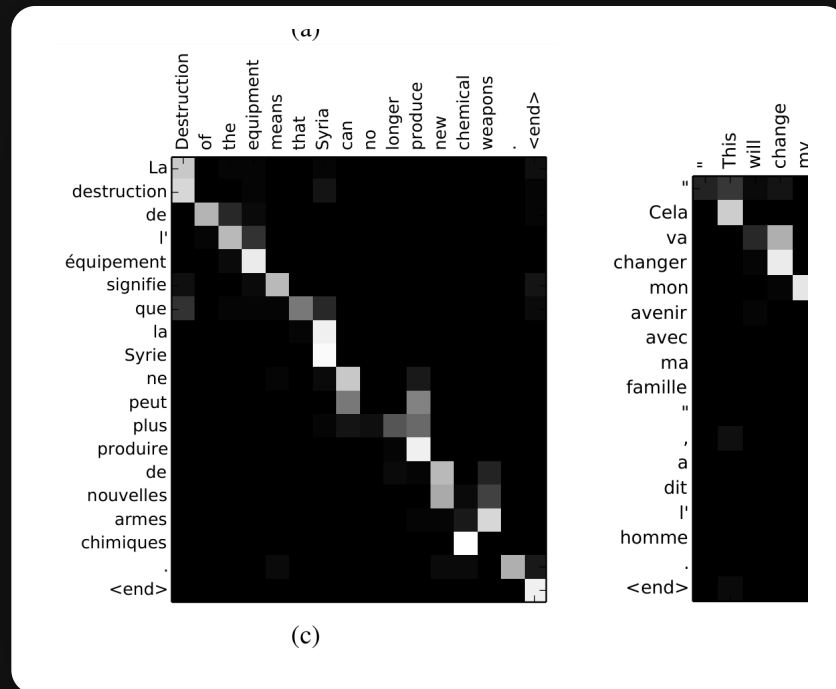


Bahdanau et al., Fig. 1 — attention weights α over a bidirectional encoder.

Reading the alignment matrix

Each cell is α_{ij} — how much target word i attends to source word j . Brighter means stronger alignment.

- The strong diagonal says translation is **roughly monotonic** — word order is largely preserved between English and French.
- But watch the **off-diagonal jumps**: where French reorders an adjective–noun pair, attention cleanly crosses to the right source word.
- A target word can attend to **several source words at once**. These weights **emerged from the translation objective**; no alignment labels were provided.



<https://arxiv.org/abs/1409.0473>, Fig 3 — EN→FR soft-alignment matrix.

A remaining limit: attention still depends on recurrence

Bahdanau gave us a powerful idea — but bolted it onto an RNN. That hybrid keeps two costs:

- The encoder and decoder are still **inherently sequential**: hidden state h_t depends on h_{t-1} , so we cannot parallelize across positions within a sentence.
- Long-range dependencies must still **propagate through many recurrent steps**, where signal and gradients decay.
- Attention is computed once per output step, layered on top of recurrence — not the primary engine.

This raises a natural question:

If attention can connect any two positions directly, **do we need recurrence at all?** What if attention *is* the model?

- Drop the RNN, let every position attend to every other position in parallel — that single move yields full parallelism and constant-length paths between any two tokens.
- That is the step we take next: **"Attention Is All You Need"** — the Transformer.

Motivation behind Transformers

Bahdanau bolted attention *onto* a recurrent encoder–decoder. <https://arxiv.org/abs/1706.03762> asked whether attention alone could suffice — what if attention is the **only** mechanism we need?

- **Give up recurrence entirely.** No more $s_i = f(s_{i-1}, \dots)$ — there is no sequential hidden-state chain to unroll.
- **Global dependencies in one hop.** Every token attends directly to every other token; long-range links no longer decay through many RNN steps.
- **Positional encodings replace order.** Since the model is permutation-equivariant, we *inject* word order with $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$ rather than reading tokens left-to-right.
- **Far more parallelizable.** All positions are processed simultaneously — training is no longer bottlenecked by sequence-length-many sequential ops.

The whole architecture rests on one reusable primitive: **self-attention**. Let us derive it from scratch.

Self-attention — the building block

Goal. Given token vectors stacked as rows of $X \in \mathbb{R}^{n \times d}$, produce X' where each token is a *content-weighted mixture* of all the others.

- Project the same X three ways: $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- **Query** q_i asks which tokens matter; **key** k_j advertises what token j offers; **value** v_j is the content carried forward.
- Because Q , K , and V all come from one sequence, this is **self**-attention.

$$q_i = x_i W_Q, \quad k_j = x_j W_K, \quad v_j = x_j W_V$$

Every output row depends on the whole sequence, while all rows are computed in parallel.

The compact form

The complete operation is Equation (1) in Vaswani et al. (2017), *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top$ — the pairwise similarity scores.
- $\sqrt{d_k}$ — the scaling factor, d_k = dimension of the keys/queries.
- $\text{softmax} \rightarrow$ weights P ; multiply by $V \rightarrow$ the mixed output.

Why divide by $\sqrt{d_k}$? For large d_k the dot products grow like $\sqrt{d_k}$ in magnitude, pushing softmax into **saturated regions where gradients vanish**. Dividing by $\sqrt{d_k}$ keeps the variance ≈ 1 , so softmax stays in its responsive range and training is stable.

Why this scales

Self-attention changes the *complexity profile* of sequence modeling:

- **Path length.** An RNN relates distant tokens through $O(n)$ steps; self-attention connects them in one $O(1)$ hop.
- **Parallelism.** Recurrence forces $O(n)$ sequential operations; self-attention uses large matrix multiplies across all positions.
- **Cost trade-off.** The $QK^\top$ matrix costs $O(n^2d)$ rather than $O(nd^2)$ for recurrence — favorable when $n \lesssim d$.

Constant path length + full parallelism explain the training advantage; quadratic sequence cost is the price.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Vaswani et al. 2017, Table 1. <https://arxiv.org/abs/1706.03762>

Multi-Head Attention

One attention does **one** kind of mixing. So run h of them in parallel, each in a smaller subspace.

- Split the model width: each head works in dimension $d_k = d_{\text{model}}/h$ (Vaswani used $d_{\text{model}} = 512$, $h = 8$, so $d_k = 64$).
- Each head gets its **own** learned projections $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$.
- Heads run independently, then we **concatenate** and mix once more with $W^O \in \mathbb{R}^{hd_k \times d_{\text{model}}}$.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Because $d_k = d_{\text{model}} / h$, total compute is roughly the **same** as one full-width head — we gain representational diversity at nearly no extra compute.

Positional Encoding: injecting order

Self-attention is a weighted **set** operation — $\text{softmax}(QK^\top / \sqrt{d_k})V$ is **permutation-equivariant**. Shuffle the tokens and the output permutes in the same way. There is no built-in notion of "first" vs "last", so we must add order explicitly.

The fix: add a deterministic **positional encoding** P to the token embeddings, so position pos and channel i are encoded by sinusoids of geometrically-spaced frequencies:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

The encoder/decoder then operate on the position-aware input:

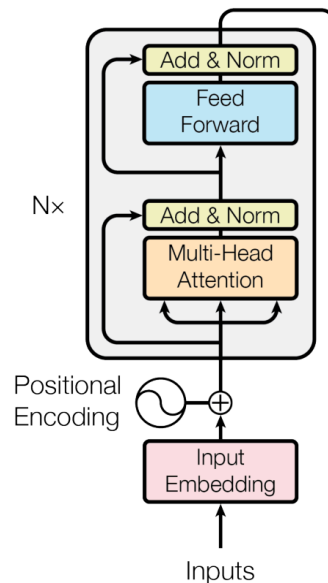
$$X_{\text{input}} = X + P, \quad X \in \mathbb{R}^{n \times d}, \quad P \in \mathbb{R}^{n \times d}$$

Same dimension d as the embeddings, so X and P simply add — no extra parameters.

The full Transformer (I)

Vaswani et al. assemble the blocks into an **encoder-decoder**. Reading bottom-up begins with token embeddings + positional encoding:

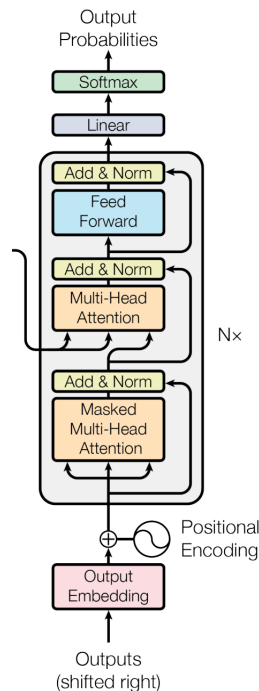
- **Encoder (left)**, $N=6$ identical layers. Each = multi-head **self-attention** (Q, K, V all from the same input) + a position-wise feed-forward network.
- **Add & Norm**. Every sub-layer is wrapped in a residual connection then LayerNorm: $\text{LayerNorm}(x + \text{Sublayer}(x))$ — stabilizes the deep stack.
- **Feed-forward (FFN)**. Applied to each position independently: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$, inner width $d_{ff}=2048$.



The full Transformer (II)

The **decoder (right)** adds a third sub-layer and a causal mask. Removing recurrence enables parallel training across positions; autoregressive generation remains token-by-token.

- **Masked self-attention.** Decoder self-attention is **causally masked** — position i may attend only to $\leq i$. Future scores are set to $-\infty$ before softmax, so generation stays auto-regressive (no peeking ahead).
- **Encoder-decoder cross-attention.** The decoder supplies Q ; the encoder output supplies K, V . This is where the target attends to the source.
- **Output head.** Final decoder states $\rightarrow$ linear $\rightarrow$ softmax over the vocabulary.



State of the art in machine translation

The Transformer **beats** recurrent seq2seq while training for a fraction of the cost.

- **Transformer base:** 65M parameters; trained on 8 NVIDIA P100 GPUs for 12 hours.
- **Transformer big:** 213M parameters; trained on 8 P100s for 3.5 days.
- **WMT 2014:** 28.4 BLEU on EN→DE and 41.8 on EN→FR, both state of the art at publication.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Vaswani et al. (2017), Table 2 — quality and training-cost evidence.

Inspectable is not the same as explained

Attention maps are appealing: they're **readable**, and a sharp, confident map feels like an explanation.

- But a peaked attention weight tells you **where** the model looked, not **why** the output is correct or what computation produced it
- Attention is one tensor in a deep residual stream; the answer is shaped by MLPs and many layers the map never shows
- The same QK^T attention machinery can support **many downstream behaviors** — including behaviors implicated in hallucination and jailbreaks — but the map alone does not explain them

A later interpretability unit — and this deck's optional appendix — asks what stronger circuit-level evidence requires.

Anchor checkpoint: from the seq2seq bottleneck to the Transformer

The through-line we built today:

- **Bahdanau** — the fixed-vector seq2seq bottleneck, fixed by an adaptive context $c_i = \sum_j \alpha_{ij} h_j$
- **Self-attention** — derived from W_Q, W_K, W_V to $\text{softmax}(\frac{QK^\top}{\sqrt{d_k}})V$
- **Multi-head** + positional encoding — parallel subspaces, order without recurrence
- **The Transformer** — SOTA translation at a fraction of the training cost

One mechanism — scaled dot-product attention — replaced recurrence and became the substrate for everything that follows.

BERT

Bidirectional pretraining

This paper anchors the shift from building the Transformer architecture to pretraining one encoder and adapting it across many tasks.

**BERT: Pre-training of Deep Bidirectional Transformers for
Language Understanding**

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language
{jacobdevlin, mingweichang, kentonl, kristout}@google.com

From one task to one model

The Transformer we just built mastered a **single task** — translation — by consuming millions of *labeled* sentence pairs. **BERT** flips that economy: learn the structure of language **once** from raw text, then adapt the same weights cheaply to almost any task.

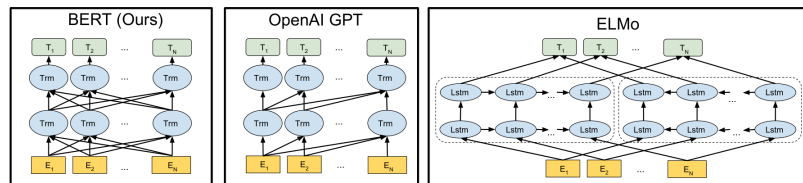
- **Labels don't scale** — a self-supervised objective takes its targets from the text itself, so the pretraining corpus can be the entire web, not one curated dataset.
- **Adapt, don't rebuild** — one pretrained encoder plus a small fine-tuned head beats bespoke per-task models — a **pretrain** → **fine-tune** recipe that reshaped modern NLP and influenced later LLM training.
- **One shared substrate** — downstream systems begin from the *same* weights, so **biases and blind spots can carry into later fine-tunes unless they are actively mitigated.**
- **The lineage** — ELMo fused separate directions; GPT fine-tuned a left-to-right Transformer; BERT made one encoder deeply bidirectional.

This shared encoder reads context from **both directions** but is not trained for efficient **left-to-right autoregressive generation** — the interface supplied by a causal decoder. Read: <https://arxiv.org/abs/1810.04805>.

Why bidirectional?

BERT's key distinction from GPT and ELMo was being **deeply bidirectional** — so why does reading both directions matter? Take **"bank"**: a left-to-right model sees only the words before it, but the evidence that disambiguates it often sits to the **right** — "river bank" vs "bank account".

- **Deep bidirectionality** lets every token, at every layer, attend to **both** neighbours simultaneously.
- You cannot get this for free with a standard LM objective — predicting the next word while seeing the future would let each word **"see itself"**. BERT's fix is masked-LM (next slide).
- ELMo's forward + backward concat is only a shallow stand-in; GPT forbids rightward context entirely.



<https://arxiv.org/abs/1810.04805>, Fig 3 — BERT (deep bidirectional) vs OpenAI GPT (left-to-right) vs ELMo (shallow concat of two LSTMs).

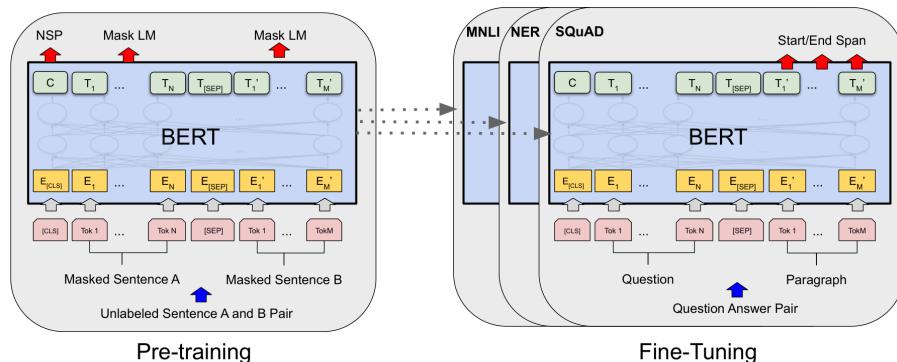
Masked language modeling

To train a bidirectional encoder, BERT corrupts the input and predicts the missing tokens — the **masked LM (MLM)**, a "cloze" task.

- Randomly select **15%** of token positions. Predict the original token at each selected position using the full left-and-right context.
- The **80 / 10 / 10 rule** for a chosen position: 80% replace with **MASK**, 10% replace with a **random** token, 10% leave **unchanged**.
- **Why not always MASK?** The MASK token never appears during fine-tuning, creating a pretrain/fine-tune **mismatch**. The random/unchanged cases force BERT to build a strong representation of **every** token, since it cannot know which inputs were tampered with.

Plus **Next Sentence Prediction (NSP)**: given sentences A and B, classify whether B truly follows A (50% true / 50% random). Teaches sentence-pair relations for QA and inference.

The two-stage paradigm



BERT, Fig. 1 — the same architecture and weights are used for pre-training (left) and fine-tuning (right).

- **Stage 1 — Pre-train:** learn MLM + NSP on unlabeled text (BooksCorpus + Wikipedia). Done once, expensive.
- **Stage 2 — Fine-tune:** reuse the same architecture and pretrained weights, add one task head, and train end-to-end on labeled data for a few epochs.

BERT is an **encoder** — it is not trained for efficient left-to-right autoregressive generation. A causal decoder provides that interface.

Limits and trust implications

BERT is powerful, but its design draws a hard line — and that line matters for how we trust it.

- **BERT is a bidirectional encoder.** It emits rich representations, but its pretraining objective is not designed for efficient left-to-right autoregressive generation.
- A **causal decoder** uses only the available prefix and exposes an explicit next-token interface — the role GPT-2 fills next.
- **Concentration of risk:** one expensive pretraining run becomes the shared substrate for many downstream tasks. Biases, gaps, or spurious correlations in that corpus **can carry into fine-tuned systems** unless later data and training counteract them.
- End-to-end fine-tuning can change the whole network, but it does not guarantee removal of inherited bias — so trust and evaluation must reach back to the **pretraining data and objective**, not just the task metric.

The pretrain-once, reuse-many-times paradigm is a strength **and** a concentrated source of fairness and reliability risk.